Predicting the Popularity of Spotify Songs Using a Neural Network

GIACOMO RUSCONI

THESIS SUBMITTED IN PARTIAL FULFILLMENT
OF THE REQUIREMENTS FOR THE DEGREE OF
MASTER OF SCIENCE IN DATA SCIENCE & SOCIETY
AT THE SCHOOL OF HUMANITIES AND DIGITAL SCIENCES
OF TILBURG UNIVERSITY

STUDENT NUMBER

2112504

COMMITTEE

dr. Murat Kirtay

prof. Samaneh Khoshrou

LOCATION

Tilburg University

School of Humanities and Digital Sciences

Department of Cognitive Science & Artificial Intelligence

Tilburg, The Netherlands

DATE

January 15th, 2024

WORD COUNT

7454

Predicting the Popularity of Spotify Songs Using a Neural Network

Giacomo Rusconi

Abstract

In the era of digital music streaming, the dynamics of song popularity have become a central concern for artists, record labels, and streaming platforms. This research sheds light on the complexity of predicting the popularity of Spotify songs using machine learning models. The dataset, sourced from Yamaç Eren Ay's GitHub repository, spans exactly a century of music and serves as the base for this exploration.

The objective of this study is to address the research question: To what extent can a Neural Network more accurately predict the popularity of songs when compared to the state-of-the-art models? To answer it, three machine learning algorithms will be used: Random Forest, Support Vector Machine (SVM), and Neural Network. They will be evaluated by considering a binary classification that divides songs between popular and unpopular.

The literature review contextualizes the study within the broader landscape of popularity prediction, drawing parallels with previous works on YouTube video popularity and Twitter hashtag virality. Most importantly, this research extends the boundaries by incorporating a Neural Network into the prediction framework, a novel approach not documented in existing literature on the 160000 songs dataset.

Results from the classification models showcase the strengths and nuances of each algorithm. The SVM excels in identifying popular songs, while the Neural Network works best when identifying unpopular songs. The Random Forest achieves a balanced performance with a high accuracy. Overall F1 between the models is around 80%, concluding that the Neural Network performs at the same level as the other State-Of-The-Art models.

Data Source, Ethics, Code, and Technology (DSECT) statement:

The dataset was acquired from the GitHub page of Yamaç Eren Ay through an online request. The obtained data is anonymised. Work on this thesis did not involve collecting data from human participants or animals. The original owner of the data and code used in this thesis retains ownership of the data and code during and after the completion of this thesis.

However, the institution was informed about the use of this data for this thesis and potential research publications.

All the figures belong to the author. In terms of writing, the author used assistance with the language of the paper. A generative language model, ChatGPT (OpenAI, 2023) was used to improve the author's original content, for paraphrasing, coding, spell-checking and grammar. No other typesetting tools or services were used.

1. Introduction

1.1 Problem statement

Nowadays, the music landscape is unprecedently accessible, among other factors because of the proliferation of music streaming platforms.

In the 1950s and 1960s, the recorded music industry grew drastically, together with other major cultural industries such as television, especially in the global north. The main record companies were electronic producers, such as RCA, EMI or Philips, which made the music market an oligopoly between these few firms. If artists wanted to be recognized for their work, they needed

to be signed by one of the record labels and there was no space or possibility for independent artists to become popular (Hesmondhalgh & Meier, 2015).
It was with the rise of punk music in the late ’70s that music became cheaper to produce, thanks to the Do-It-Yourself mentality of these bands, that subsequently led to the indie movement in the ‘90s when it got so important that the United Kingdom had to create “Indiependent” charts to promote music that was beyond the usual top 50 (Hesmondhalgh & Meier, 2015).
In the late 2000s, the digitalization of music forced artists and record labels to reconsider how to promote their music. People did not need to produce music in expensive studios anymore, they did not need to press their music in physical form, and because of the internet, they could just upload their songs online and they would be immediately available to everybody (Ogden et al., 2011). Although it could have potentially been seen as a positive development, creating an easier way for everybody to enter the music market, it should be noted that in 2023 the music streaming revenues were a third of the total revenues of the music industry, more specifically 66.9% of a total of 26.2 billion (IFPI, 2023).
While this accessibility is great for the listeners, it creates a new challenge for artists and record labels that have to fight for recognition in an oversaturated environment. Artists now have to fight with a huge amount of competition, and record labels have to consider the incognita of the algorithm of the platform where they are promoting their artists’ music. These algorithms change within different music streaming platforms and are considered black boxes, meaning that people do not know how they work and how they evaluate whether a song is popular, and they select which song should be shown to more people depending on different parameters (O’Dair & Fry , 2020).
This shifts the goal for artists and record labels from creating appealing music and promoting it to people to understanding what are the factors for which these algorithms consider a song popular and suggest it to more listeners since the profit of a song depends directly on how many people listen to that song.
As will be shown in the literature review chapter, many researchers have already used machine-learning techniques to understand which features were the most influential to make a song popular, but this thesis does not focus on this aspect. The major gap in the research on the topic is the fact that to the best of my knowledge, none of them used a Neural Network in order to predict a song’s popularity. At the moment of the writing of this thesis, the State-Of-The-Art (SOTA) models are the Random Forest and the Support Vector Machine, which will be used throughout this thesis to compare the performance of the Neural Network.
This project is situated at the nexus of data science and music business, and it aims to find out whether it is possible to create a more accurate method of predicting whether a song will be a chart-topping hit or not with the use of a neural network model, providing such insight to artists and record labels in order for them to navigate the intricate landscape of music streaming.

1.2 Research Question

To what extent can a Neural Network more accurately predict the popularity of songs compared to the current state-of-the-art models?

1.3 Societal & Scientific Relevance

Considering the scientific context, many papers used this specific dataset and predicted the popularity of songs, but none of them implemented the use of Neural Networks while investigating this. The papers with the best-performing models used either a Random Forest method or a Support Vector Machine (SVM), hence in this thesis, the performance of the Neural Network will be compared to the results demonstrated by the other two models. This will be addressed more in-depth in the “Literature Review” chapter.
In the eventuality that this research will enhance the current State of the Art (SOTA), the results could be also used to create better recommendation systems, mainly to let users be exposed to a more diverse range of music that aligns with their preferences but may not be widely known yet. A better classification model can also improve the personalisation aspect of recommendation systems, being more advanced in getting the nuances of each preference. Advances in classification could also be applied to other media unrelated to music, such as movies or social

media posts.
Societally, better recommendation systems can be beneficial for listeners, being able to discover new music that aligns at best with their taste, and from the artists' perspective, they can get their music delivered to a more appropriate and tailored public. The record labels will directly benefit if their artists get more listens and become more popular, and lastly, the streaming platforms could use this improvement in technology to retain their users and provide them with a better experience of music discovery.
Since this thesis is based on the publicly available 160000 songs dataset (Ay, 2020), the results of this research will be transparent and other researchers are welcome to replicate the findings.

2. Literature Review

In recent years there has been a surge in publications that focus on popularity among different media using different machine learning techniques. In 2017 Trzcinski and Rokita predicted the popularity of YouTube videos using the Support Vector Regression model (Trzcinski & Rokita, 2017) and another team of researchers (Ma et al., 2013) applied five different machine learning models, namely, Naïve Bayes, k-nearest neighbours, Decision Trees, Support Vector Machines, and Logistic Regression, in order to accurately predict whether or not a hashtag will become viral on Twitter.
Considering the specific case of music media, papers discussing how to predict whether or not a song will be a hit appeared as early as 2005 when Logan & Dhanaraj classified songs considering acoustic and lyric data using a Support Vector Machine (Dhanaraj & Logan, 2005). Given the time that this research was published and the minimal dataset used of only 1700 songs, they reached results suggesting that the popularity of a song is not a random event, but could depend on specific characteristics of a song, with the highest average ROC (Receiver Operating Characteristic) value obtained combining acoustic and lyrics data of 0.69.
In 2015, Singhi and Brown published a related work, this time focused more on the role of rhyme, syllables and meter features on a dataset of around 7000 songs, being able to correctly classify 50% of the hits and misclassify 12.8% of flops (meaning, unpopular songs) when using their broader definition of a flop (Singhi & Brown, 2015). A significant contribution of this paper was the subdivision of songs into hit and flop. While many modern researchers, especially those who used the 160000 Spotify API dataset, used the given popularity metrics as a range between 0 and 100, this thesis used a similar approach to the one of Singhi and Brown. They considered a song as popular if it was within the Billboard Year-End Hot 100 singles chart between 2008 and 2013, deleting duplicate songs repeated across two years, and considered a song a flop depending on 4 different definitions, the broadest being a song of a “hit artist” that did not reach the definition of a hit song, and the narrowest being songs that were published as singles but did not enter the Billboard chart of their specific year. For the scope of this thesis, the songs are considered as either hits or flops or similarly, as popular or unpopular, pursuing the overall concept of Singhi and Brown (2015) but considering reaching a balanced dataset as the main reason for the cut-off. This last part will be further explained in the paragraph “Feature Extraction”.
Other researchers combined the use of language processing with audio analysis of spectrograms and found out that by considering the musical part of a song it was possible to predict even more closely whether or not that song would be a hit or not, reaching a percentage of accuracy of 83.02% when considering their best performing model (Martin-Gutierrez et al., 2020).
In recent years, with the availability of datasets such as The Million Song Dataset (Bertin-Mahieux et al., 2011) and the 160000 Spotify API dataset, which links for the download is available in Appendix 1 of this thesis (Ay, 2020), many publications attempting to predict the popularity of songs have been produced, all of them using different types of machine learning algorithms such as Logistic Regression, Decision Trees, Naïve Bayes, Lasso Regression and Random Forests.
One example is the one of Agha and Krishnadas, who, while using a limited number of sample songs and having the results of 52% using their best-performing model, a logistic regression algorithm, were among the first people to ever use the information provided by the Spotify API in order to answer the question of popularity prediction in music (Agha & Krishnadas, 2020).
Another notable example is the paper of Araujo who, while choosing a dataset smaller and having a different approach when considering which songs were popular and which were unpopular,

brought two ideas that will be used throughout this thesis: the first one is the use of a Support Vector Machine (SVM), specifically using a radial basis function (RBF) kernel, that was the best performing algorithm in his paper, reaching an accuracy of 85%; and the second was the concept of binarizing the popularity of songs, discarding the range from 0 to 100 provided by the Spotify API and considering a song as popular or unpopular, similarly to what had already been done by Singhi & Brown (Araujo, 2020). While this paper will be used to compare the performance of this thesis' SVM, it is important to note that his binarization between popular and unpopular songs is different from the one used by Singhi and the one that I will perform. Araujo's paper considered songs that were in the Top 50 as popular and songs that were in the Viral 50 as unpopular. Some researchers could argue that both songs that are in the Top 50 and Viral 50 could be considered popular, but Araujo quoted a paper by Herremans saying that they utilized the same method, so it will still be considered a valid methodology throughout this thesis (Herremans , 2014).
With the publication of the 160000 songs dataset in April 2021, many researchers interested in the intertwined role of music prediction and data science utilized this dataset, mainly in the sphere of Exploratory Data Analysis (Otuokere et al, 2021).
Another important paper is the one of Essa, who used the same dataset that had been used during this research and worked with a vast number of algorithms, specifically four regression models and 7 classifiers. Of all these models, Random Forest is the best-performing one when considering accuracy (89.54%), although other models produced very similar results. Given that the dataset used is the same, and the only differences are in the data cleaning process, it is expected that the results of this paper will be comparable to the one of the current thesis (Essa et al, 2022).
To the best of my knowledge, as of the moment of the writing of this thesis, there are no researchers that have applied a Neural Network model to this dataset in order to predict the popularity variable. This research aims to fill this void in the literature and to provide a broader knowledge of the application of machine learning models in the field of music popularity prediction.

3. Methodology

3.1 Dataset Description

In order to build the machine learning models that will predict a song's popularity, a dataset with exactly 169907 songs extracted from Spotify will be used. It was created by Yamaç Eren Ay and it consists of 169907 rows (songs) and 19 columns (features) saved in a CSV file and ranging from the year 1921 to 2020, covering exactly one century of music (Ay, 2020).
All of these variables can be sub-divided into three macro-categories:
The first group are Confidence Measures, where we can find values such as liveness, instrumentalness, acousticness or speechness. All of these variables are of type float and range from 0 to 1, showing respectfully how much the Spotify API think a song is played live, instrumental, acoustic or spoken words.
Another category of floating type variables made of a range between 0 and 1 are Perceptual Measures, which are variables that depend on many perceptual information combined, but for the sake of simplicity are reduced to ranges from 0 to 1. For example, the variable “Energy” represents the intensity of a track, where an energy level close to one means that that song will be loud, noisy and fast.
The last category is the most straightforward, and it is the one for Music Descriptors. All of the features in this category are objective music attributes, such as Tempo, calculated in BPM (beats per minute), Key or Year of release (De La Rosa, 2021).
For a more in-depth description of each one of the 19 features, a complete list based on Spotify's documentation for its API is provided (Web API Reference, Spotify for Developers, n.d.).

Feature	Description
Id	A unique identifier for each track, for example, "2ZywW3VyVx6rrrrX75n3JB".
Name	Represents the song's title in brackets, followed by information regarding whether it is live, a remaster, a remix or similar.
Artists	The name of the artist or artists that participated in that track.
Duration_ms	The length of the song in milliseconds (ms).
Release_date	The track's release date is in the format MM/DD/YY. When this data isn't available, it just shows the year of release.
Year	The year in which the track was released.
Acousticness	A confidence measure from 0.0 to 1.0 of whether the track is acoustic. 1.0 represents high confidence that the track is acoustic.
Danceability	It describes how suitable a track is for dancing, with a value of 0.0 being the least danceable and 1.0 being the most danceable. It is based on a combination of musical elements including tempo, rhythm stability, beat strength, and overall regularity.
Energy	A measure that ranges from 0.0 to 1.0 and represents a perceptual measure of intensity and activity. Typically, energetic tracks with a score close to 1.0 are faster, louder and noisier.
Instrumentalness	It predicts whether a track contains vocals or not. Songs with a value close to 0 are considered to be very vocal. Vocal parts such as "aaah" or "oooh" are not considered vocals in the context of this feature.
Liveness	It detects the presence of an audience in the recording. A higher value of this variable represents a higher chance that the song was performed in front of a live audience.
Loudness	It shows the average loudness of a track in decibels (dB) in a range from -60 to 0. The loudness values represent an average across the entire track.
Speechiness	It is a variable that ranges from 0.0 to 1.0 and it detects the presence of spoken words in a track, and could therefore be considered the opposite of the feature "Instrumentalness".
Tempo	The overall estimated tempo of a track in beats per minute (BPM).
Valence	A measure from 0.0 to 1.0 that describes the musical positiveness conveyed by a track. Songs with a valence value close to 0 are considered more negative, giving feelings of sadness or anger.
Mode	A binary value that explains the melodic content from the modality of a track. Songs with a variable Mode equal to one are considered to be in a major key, while those with a value of 0 are in a minor key
Key	The estimated overall key of a track from the Pitch Class notation. For example, 0 = C, 1 = C♯/D♭, 2 = D, and so on until 11. If no key was detected, this variable gets a value of -1
Popularity	It shows a track's popularity in a range between 0 and 100, where 100 is the most popular based on Spotify's algorithm.
Explicit	It is a Boolean value that represents whether or not the track has explicit lyrics, having a value TRUE when the song is explicit.

Table 1: Description of the variable of the dataset (Web API Reference, Spotify for Developers, n.d.)

3.1.1 "Popularity" Parameter

The column that is most interesting for this study is certainly the "Popularity" one which, while being a simple range of integers between 0 and 100, is probably the hardest to define. It is hard to objectively measure how popular a song could be, but since the dataset of Yamaç Eren Ay was derived from the Spotify API (Ay, 2020), this paper will consider Spotify's definition of popularity. Quoting the documentation page of Spotify API, "The popularity is calculated by algorithm and is based, for the most part, on the total number of plays the track has had and how recent those plays are." (Web API Reference, Spotify for Developers, n.d.).
As shown, Spotify doesn't want to fully disclose the specifications of how its algorithm works but provides us with an insight into the importance of recent time plays. The number of total plays that a song has is not the only criterion to consider when evaluating a song's popularity. Furthermore, Spotify's API states that duplicate songs, meaning songs that appear both as singles and within an album, are rated independently. For the scope of this research, we will be deleting such duplicates, but this will be further explained in the following paragraph.

3.2 Preprocessing

3.2.1 Data Cleaning

Before dealing with the models' development, some features had to be deleted from the dataset. First of all, the variable "ID" was dropped, since it represents a unique identifier for that specific song, which is not considered to be helpful for the sake of this research.
Another variable that was dropped soon in the process was the "Release Date" column, which provides information about the day, month and year of release of a particular song. Although it could have led to some interesting insights, most of the data within this variable are the same as the variable "Year", meaning that they lack data about the day and the month. More specifically, the total number of entries that have a full "Release Date" variable is 66842 out of 169907, which is considered to be a proportion too small to be considered relevant.
After this step, some duplicate rows were deleted. As stated previously, Spotify considers the same song published as a single or together with the album as two separate entries, even though the audio file is the same. For this reason, all of the duplicates that had the same "Name" and "Artist" values have been deleted, bringing the total number of entries from 169907 to 156607. In a preliminary step of this thesis, it was hypothesised to keep the entry with the highest level of popularity and drop the duplicate with a lower popularity score, but after careful consideration, it has been chosen to drop these duplicates without considering their popularity score, in order to have a more randomised and neutral approach to the data cleaning.
This process does not completely delete all the versions of the same song though, as can be understood from the example of the song "40" by the band "U2" showcased in Figure 1:

name	artists
"40"	['U2']
"40" - Live	['U2']
"40" - Remastered 2008	['U2']
"40" - Remastered 2008	['U2']

Figure 1: Example of different audio files from songs with the same name and artists

There are four entries in total: 40, 40 – Live and two entries of 40 – Remastered 2008. Considering the sound features, 40 – Live has, intuitively, a higher value of liveness, while the differences between 40 and 40 – Remastered 2008 are more subtle but still present (for example, a different mix in order to deliver a better audio experience with the current music equipment). For the sake

of this research, I decided to remove only the duplicates that come from the same audio file, keeping different versions of the same song if necessary. Then, the variables "Name" and "Artists" were removed. This choice has been made due to the time limitations of this research but for future development, it would be interesting to explore how the inclusion of the variable "Artists" would influence the prediction of the popularity score. This could be supported by the argument that already famous artists are statistically more likely to have a hit song when compared with a relatively unknown musician, but for the time being this is beyond the goals of this research.

3.2.2 Understanding Majority Class

When checking the distribution of the Popularity variable, it was discovered that the number of songs with a Popularity score equal to 0 was much higher when compared to the other scores. The graph shown in Figure 2 shows the distribution of the popularity variable before the undersample.

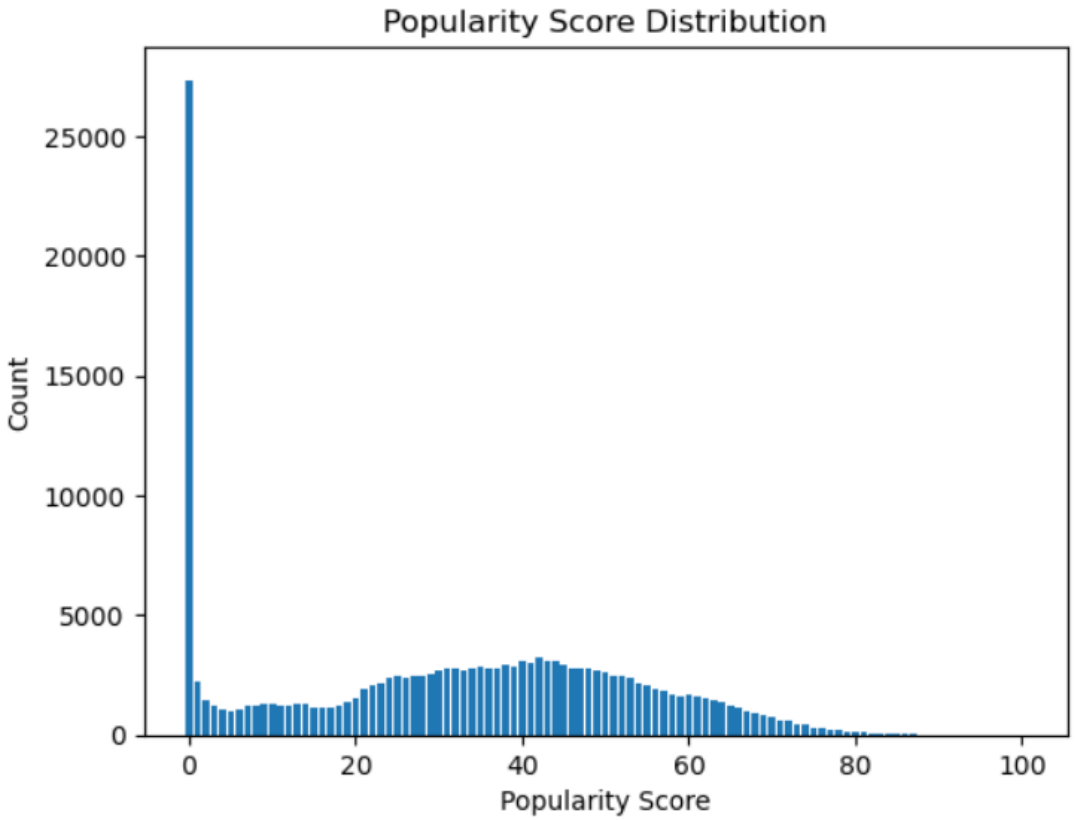

Figure 2: Distribution of the popularity variable before the undersampling step

In the dataset used during this research, the majority class for the feature "Popularity" is "0", with 25884 songs, while the second-highest popularity score, namely, the score "42" has 3015 songs, with a proportion of more than 8:1. This could lead to several problems, such as having the model to be biased towards the majority class, meaning that in the context of our thesis they would classify songs as not popular more often than they should. Having a balanced dataset can prevent overfitting and, because of the size of the dataset used during this research, undersampling has been considered more adequate to use than oversampling, being able to reduce the dataset size enough to reduce training time but not too much to lose an important amount of information.
Because of these reasons, the number of songs with a Popularity score of 0 has been undersampled to the same amount of songs with the second-highest popularity score, randomly deleting 22869 songs from the dataset and making the final number of songs used in this thesis 133738. After this data-cleaning process, the distribution of the popularity variable looked like the following graph.

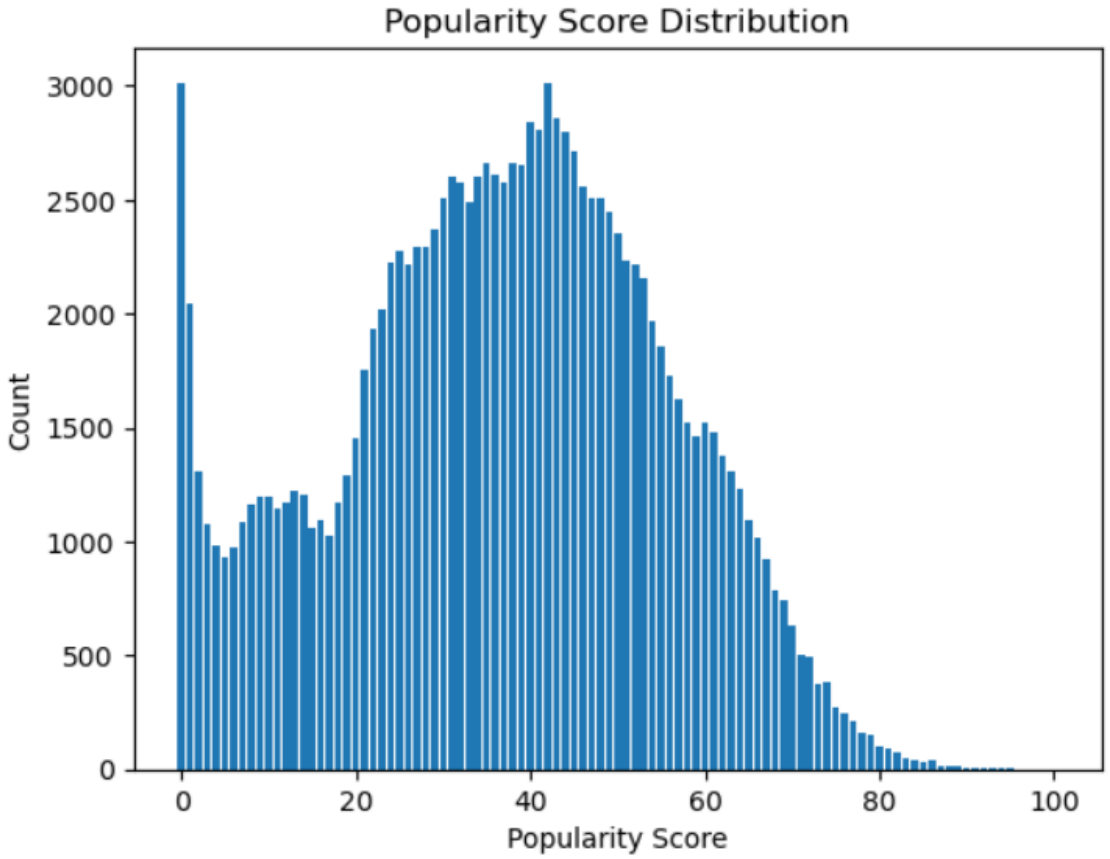

Figure 3: Distribution of the popularity variable after the undersampling step

3.2.3 Feature Extraction

As stated previously, the popularity score is given by an undisclosed algorithm of Spotify and because of this uncertainty, it is not deemed necessary to regress the exact score of a song, but rather understand whether that song could be considered popular or not*, similarly to what other researchers such as Singhi or Araujo have been done.
In order to do this, a new binary variable called "popularity_binary" has been created considering a cut-off at the Popularity score of 40. This variable takes the original "popularity" variable and simply shows that a song is popular by giving a value of 1 when the popularity score was equal to or above 40, therefore considering a song as popular, and giving a value of 0 when the popularity score was lower than 40, meaning that those songs were considered unpopular. The cut-off value of 40 has been chosen after careful testing and keeping the two binary states as balanced as possible. After the creation of this variable, the songs that are considered popular, hence having a "popularity_binary" score of 1, are 72129 and those considered unpopular, having a "popularity_binary" score of 0, are 61609. With a proportion of 1.17:1, the variable "popularity_binary" is more balanced than before. For the sake of this research, the dataset will still be considered slightly imbalanced, using the F-1 metric during the models' evaluation.

3.3 Machine Learning Pipeline

The following figure represents the Machine Learning Pipeline used throughout this thesis. For more in-depth information about the steps, it is recommended to read the according paragraph.

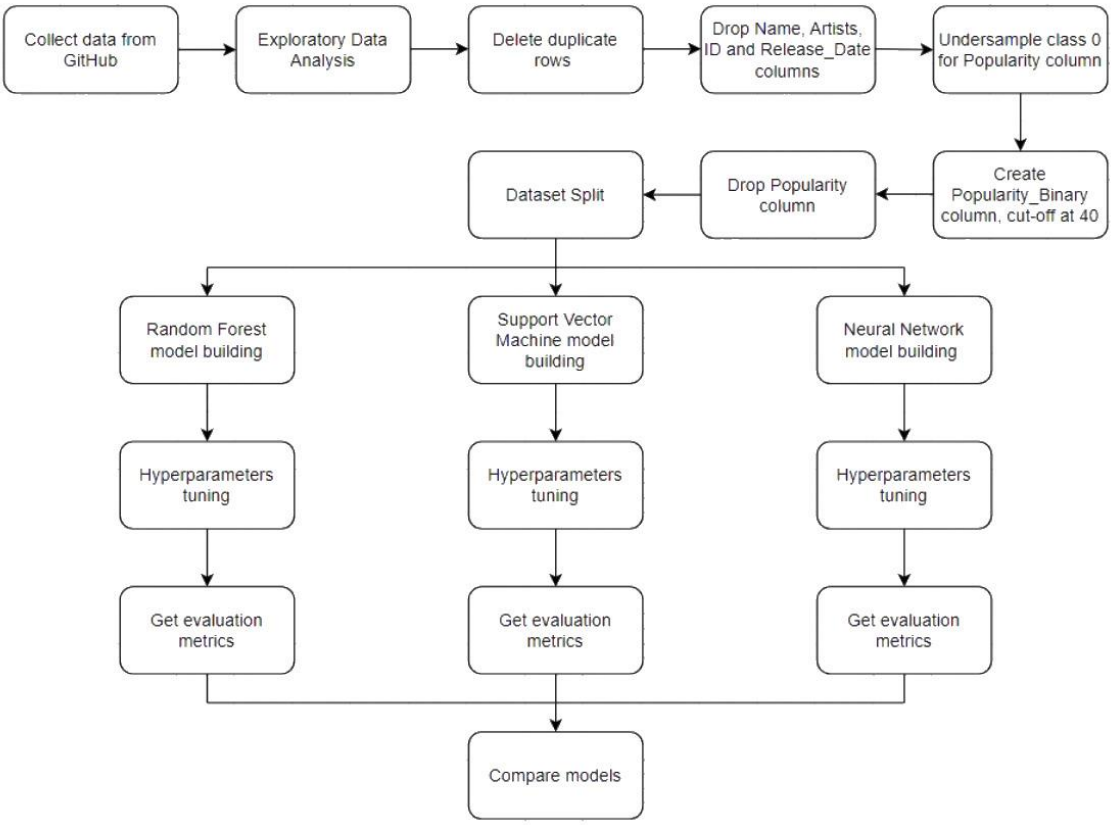

Figure 4: The graph showcasing the Machine Learning pipeline

3.4 Models

During this research, three different machine-learning algorithms were used as described in the following paragraphs: Random Forest, Support Vector Machine and Neural Network.
The Random Forest will be used as a baseline and will be compared with the Random Forest model utilized by Essa, the SVM will be compared to the one used in Araujo's paper and the Neural Network is an experimental method that, to the best of my current knowledge, was never used with this particular dataset.
These tree models will all predict the variable "popularity_binary", which makes this research a binary classification problem.
All the models had their hyperparameters tuned as shown in each model's paragraph and they were chosen using a random search. The use of random search was chosen for technical reasons, given that the machine used to process these models was not powerful enough for more advanced algorithms such as Grid Search or Bayesian Optimization. For the same technical reasons, this paper used parallelisation, which is a technique that splits the computational work across multiple CPU cores and makes the code run smoother (8.3. parallelism, resource management, and configuration Scikit, n.d.).
The split between training, validation and testing sets has been made through cross-validation using the k-fold method, more specifically using a k parameter of 5.
Once trained, these three models will be evaluated using precision, recall and F1 from both classes and accuracy for an overall evaluation. These evaluation methods have been chosen because they were the standard used by previous researchers, in order to be able to draw some similarities and compare this thesis with past research.
Since this dataset was extracted from Spotify's API, future researchers are welcome to test these models on out-of-sample data, for example by analyzing more recent songs. This could be applied smoothly in the case that Spotify will not change its evaluation algorithm for any of the variables of this dataset used to predict popularity.

3.4.1 Random Forest

Random Forest, introduced by Breiman in 2001, is a versatile machine-learning algorithm that works by combining multiple decision trees, each trained on a distinct subset of the data. The

final prediction is chosen by considering the majority class among the individual trees. The main characteristic of this ensemble learning-based classifier is a strong emphasis on randomization, achieved through different mechanisms. Firstly, it uses a parameter represented by a random vector. Secondly, it selects features in a randomized manner. Additionally, it considers a random selection of a subset from the dataset as the training set. This intentional randomness contributes to the algorithm's robustness, making it able to work consistently in diverse scenarios (Mohapatra et al., 2020).

Although its conceptual simplicity, Random Forest remains a widely employed algorithm due to its demonstrated greatness in accuracy and reduced overfitting compared to other commonly used algorithms (Liu et al., 2012). Because of its natural flexibility, this model is able to perform at high levels with various types of predictors, regardless of them being continuous or categorical (Mohapatra et al., 2020).

The loss function utilized in this research is the Gini impurity, the standard one for the Random Forest model of the Sklearn library. This measurement calculates how often the elements in the dataset would be mislabeled if the labels were assigned at random.

Within the context of this thesis, Random Forest has been chosen as the baseline model because it was the best-performing method in the research of Essa and is considered to be the current state of the art (Essa, 2022).

A total of 7 hyperparameters were tuned, as showcased in the following table that offers a view on which hyperparameters were tuned, which values were optimal and a brief description for each.

Hyperparameter	Chosen Value	Description
n_estimators	200	Number of trees in the forest.
min_samples_split	5	Minimum number of data points placed in a note before the node is split.
'min_samples_leaf'	4	Minimum number of data points in a leaf node.
'max_features'	sqrt	The number of features to consider when choosing the best split. In this case, is sqrt(n_features).
'max_depth'	10	Maximum depth of a tree.
'class_weight'	balanced	The weight associated with each class. In this case, the weight is inversely proportional to the frequency of the class in the input.
'bootstrap'	False	Bootstrap was not used, the whole dataset was used to build each tree.

Table 2: Hyperparameters tuned for the Random Forest model

3.4.2 Support Vector Machine

Introduced by Boser et al. in 1992, the Support Vector Machine is an algorithm created for model prediction using training data to forecast target values for test data based on their variables (Hsu et al., 2010). To explain the mechanism behind SVM, it aims to find a hyperplane that maximally separates data points into distinct classes. This hyperplane is strategically positioned in order to maximize the distance between the hyperplane and the closest data point of each class.

One of the main features of SVM is its capability to handle non-linear data. It can achieve this by transforming the original feature space into a higher-dimensional space and creating a linear hyperplane to effectively separate the data. This makes SVM particularly useful for addressing classification problems (Hsu et al., 2010).

In the context of this study, the classification problem involves a vector of 15 dimensions, with each dimension representing a musical feature. The SVM algorithm, with its capacity to delineate decision boundaries in multi-dimensional spaces, aligns with the intricacies of this dataset.

Similar to the Random Forest, the loss function for the SVM is the default for the Sklearn library, the Squared Hinge. Because of this, the goal of the SVM will be to find the best hyper-plane that trade-off maximizing the margin and minimizing the Squared Hinge.

Because the SVM bases its decisions on the distance between the data points, it is sensitive to the scale of the input. To overcome this, all the independent variables were scaled using the StandardScaler command.
During this study, 5 hyperparameters were tuned, as shown in the following table.

Hyperparameter	Chosen Value	Description
kernel	rbf	The type of kernel used by the algorithm. In this case, the Radial Basis Function (rbf) kernel has been chosen as the optimal one.
gamma	scale	The value of the gamma coefficient for the rbf kernel. Here, it uses the formula 1 / (n_features * X.var()).
degree	3	A parameter used by the "poly" kernel. Because here the rbf was chosen, this parameter is ignored.
class_weight	balanced	The weight associated with each class. In this case, the weight is inversely proportional to the frequency of the class in the input.
C	1	The regularization parameter, higher C means a weaker regularization.

Table 3: Hyperparameters tuned for the Support Vector Machine model

3.4.3 Neural Network

Considering the Neural Network, this study will use more specifically the Multi-Layer-Perceptron. It is a classifier originally conceptualized by Warren McCulloch and Walter Pitts in 1943. The MLP, an extension of the traditional perceptron model, was developed to surpass the limitations associated with linearly separable problems (McCulloch & Pitts, 1943). The architecture of the MLP is characterized by multiple layers of connected nodes, known as neurons or perceptrons. The network is made of an input layer, one or more hidden layers, and an output layer. Each connection between nodes of different layers has an assigned weight, and each node within the network has an activation function to the weighted sum of its inputs, giving the model its non-linearity properties.
The MLP Classifier, in this case, implemented through the libraries Scikit Learn, involves a training process where the model learns the optimal weights for the most accurate predictions (Gardner & Dorling, 1998). This is done through a backpropagation algorithm, which adjusts the weights based on the error in the output compared to the expected result. The activation function, which in the case of this research is the logistic sigmoid function, introduces complexity to the model, making it able to capture intricate patterns within the data.
Because of its ability to adapt to different and complex datasets, it is considered to be a valuable choice for the scope of this thesis.
The loss function for this model is the cross entropy, which is the only one used in Sklearn.
Similar to what has been done with the SVM, the Neural Network had its independent features scaled. This decision was given by the fact that if the scales between different variables are uneven, the gradient could vary widely, making the update of the weights uneven too.
Before computing the Neural Network, 4 hyperparameters have been tuned, as shown in the following table.

Hyperparameter	Chosen Value	Description
learning_rate	invscaling	The learning schedule for weight update. Here, gradually decrease the learning rate for every step "t"
hidden_layer_sizes	(100,)	Number of neurons in the nth hidden layer.
alpha	0.001	Strength of the L2 regularisation term
activation	logistic	The activation function for the hidden layer. Here the logistic sigmoid function was used.

Table 4: Hyperparameters tuned for the Neural Network model

3.5 Performance Metrics

The classification report function of Scikit Learn has been used to evaluate the performance of the models that will be developed. This function outputs for each model the precision, recall and F1 score of each class and the accuracy of the model.

3.6 Software Components

In order to set up the models and compute the data, it will be used version 3.9.13 of Python. The libraries that have been used during the process of this paper are NumPy (Harris et al, 2020), Matplotlib (Hunter, 2007), Pandas (McKinney, 2010), seaborn (Waskom, 2021) and the following list of different functions from the Scikit Learn library: MLPClassifier, classification_report, train_test_split, cross_val_predict, cross_val_score, GridSearchCV, RandomizedSearchCV, StratifiedKFold, RandomForestClassifier, SVC, StandardScaler, accuracy_score (Pedregosa et al, 2011).

4. Results

In this chapter, the results of the performance of the models will be presented.
All of the results were reached using the classification report function of Scikit Learn and rounded to the second decimal point percentage.

Performance Metrics	Neural Network	Support Vector Machine	Random Forest
Precision (Class 0)	80.91%	**82.04%**	81.51%
Recall (Class 0)	**90.70%**	88.26%	89.77%
F1-Score (Class 0)	**85.53%**	85.03%	85.44%
Precision (Class 1)	**87.32%**	84.91%	86.41%
Recall (Class 1)	74.95%	**77.37%**	76.16%
F1-Score (Class 1)	80.66%	**80.97%**	80.96%
Accuracy	83.45%	83.24%	**83.50%**

Table 5: Comparision between different performance metrics of all the three methods used in this thesis

4.1 Models Performance

As previously mentioned in the “Models” paragraph, all three models were tested using a 5-fold cross-validation.
These results showcase the ability of each model to predict a song’s popularity, with minor differences among the various performance metrics and having the recall for popular songs in the Neural Network as the lowest scoring measure in the whole study, still reaching a percentage of 74.95%.
Considering the results demonstrated in the tables above, Random Forest had a higher accuracy than the other models, while never being the best performer in any of the other metrics.
The SVM was the best-performing method when considering the classification of popular songs, although having only a .01% higher F1 score than the second-best model for this parameter, the Random Forest.
Lastly, the Neural Network performed best when identifying unpopular songs, with an F1 score for the class 0 0.09% higher than the Random Forest, the second-best performing model for this metric, and an F1 score for the class 1 0.31% lower than the SVM, the best performing model for this metric.

A heatmap for each model's confusion matrix is presented in Appendix 2.
The next chapter will give a more comprehensive explanation of the findings, highlighting observations and interpretations regarding the use of the Neural Network in this dataset.

5. Discussion

The results of the classification models provide insights into the effectiveness of different machine learning algorithms in the prediction of the popularity of songs. Overall the models exhibited results that are similar to those obtained from other researchers on the same topic.

5.1 Random Forest Discussion

The Random Forest demonstrated a higher accuracy than the other two models, although performing slightly worse in all the other evaluation metrics. The following table will compare the results obtained in this thesis with those obtained by Essa when using the Random Forest. Their paper did not provide the metrics for class 0 (Unpopular songs), so in this table, they will be substituted with "Not available".

	Rusconi's Random Forest	Essa's Random Forest
Precision (Class 0)	81.51%	Not available
Recall (Class 0)	89.77%	Not available
F1-Score (Class 0)	85.44%	Not available
Precision (Class 1)	**86.41%**	86%
Recall (Class 1)	**76.16%**	69%
F1-Score (Class 1)	**80.96%**	76%
Accuracy	83.50%	**89.54%**

Table 6: Comparision between this thesis' and Essa Y.'s Random Forest models

The Accuracy of the Random Forest used by Essa is higher than the one used in this paper, reaching an Accuracy score of 0.8954. For all the other measures, the Random Forest model used in this thesis reached more robust results and higher scores. While in the case of Precision, it is just slightly higher, in others, such as for the Recall, this paper's Random Forest reached a score of 76% against the 69% of Essa's model.
It is important to consider that the methodology used by Essa's paper was slightly different. The most important difference is that Essa considered popularity as a range of scores between 0 and 100, while this thesis considered a binary variable that would just divide songs into Popular and Unpopular. The Random Forest models themselves had also different values for their hyperparameters, generally with the model of Essa being more complex, having for example 400 trees against the 200 used in this thesis' model or a maximum depth of a single tree of 20 against the value of 10 of the present work.

5.2 Support Vector Machine Discussion

Considering the Support Vector Machine, its best performance was in regard to the labelling of Popular songs. The following table will compare the results of the model obtained from this thesis to those of the model used in Araujo's paper (2020). Similar to what happened with the Random Forest model, Araujo's paper did not provide any metrics for class 0 (Unpopular songs) and because of this they will be substituted with "Not available".

	Rusconi's SVM	Araujo's SVM
Precision (Class 0)	82.04%	Not available
Recall (Class 0)	88.26%	Not available
F1-Score (Class 0)	85.03%	Not available
Precision (Class 1)	84.91%	**96.5%**
Recall (Class 1)	**77.37%**	77%
F1-Score (Class 1)	80.97%	**85.69%**
Accuracy	83.24%	**85.11%**

Table 6: Comparision between this thesis' and Araujo's Support Vector Machine models

Overall, the model used by Araujo outperformed the one considered in this thesis. While some metrics such as Accuracy or Recall were relatively close to each other, with the model used in this thesis being slightly higher when considering the Recall, the F1 of the SVM used by Araujo is almost 0.05 higher than the one of this thesis and the Precision is 0.12 higher.
This might be given by different factors and differences within the two methodologies. A major difference is within his database, which was built in a different way. Although using the same variables provided by the Spotify API, he considered a period from November 2018 to July 2019 and labelled as Popular the songs that were in the Top 50 and as Unpopular those that were in the Viral 50, this method being already covered in the literature review. Additionally, he transformed the variables of danceability, energy, speechiness, acousticness, instrumentalness, liveness and valence into binary values, considering a cut-off value of 0.5 when labelling them as either 0 or 1, eliminating some degrees of complexity in his dataset. In the paper of Araujo, it is not specified how his Support Vector Machine was hyperparameter tuned, but he used the same kernel considered in this thesis, the RBF kernel.

5.3 Neural Network Discussion

Lastly, the Neural Network outperformed the other models used in this thesis, but without having a substantial difference when compared to the others. It demonstrated the highest score for F1 and Recall when considering Unpopular songs, while still showing the highest score for Precision when considering Popular songs. It was the second-best model considering the Accuracy metrics, with a difference of just 0.0005 from the highest-scoring model, the Random Forest.
Considering the two F1 metrics, the Neural Network was 0.09% higher than the second-best model for class 0, but was the worst-performing model for class 1, showing a 0.31% difference from the best-performing one. The highest difference in score between the Neural Network and the second-best model was considering the Recall for class 0, having a difference of 0.93% from the Random Forest model.
Within the context of this thesis, the Neural Network demonstrated to be a valid alternative to the other methods that are currently considered the State Of The Art, being capable of even performing slightly better than them in some metrics.

5.4 Final Discussion

The findings of the research suggest that all three models offer valuable predictive capabilities, each of them having strengths from different perspectives.
The minor variations in the models' performance could derive from the complexity of predicting the subjective concept of songs' popularity. The core nature of musical preferences, influenced by a very large number of factors, makes the task inherently challenging. Despite the challenge, the results affirm the viability of employing machine learning models for predicting a song's popularity. The Neural Network model can be used as an alternative to the current state-of-the-art by artists, music labels and streaming platforms in order to create recommendation systems to satisfy the listeners' needs. Future research on the topic can further explore the potential of this method.
In conclusion, the Random Forest, SVM, and Neural Network models contribute unique strengths to the prediction task, while still producing satisfactory and similar results.

5.5 Limitation & Future Research

Future research could further develop the topic in different ways. As stated previously, a possible approach would be working towards understanding whether the inclusion of the variable "Artists" into the dataset could lead to more accurate results. An artist with an already substantial fanbase could potentially get a certain number of plays every time they upload a new song, meaning that the popularity of an already notable artist's new song could be higher than the popularity of a relatively unknown artist's song. A possible approach could be one of averaging the popularity score of all of the songs of an artist present in the database and using that number as a popularity score for the artists themselves, considering the importance of the

artist's fame as a key variable in the popularity of a future song.
Another way to further improve this research would be a more thorough grid search, which was not possible in this paper due to the technical limitations of the hardware device used for the computation of the dataset. Considering similar technical issues, with more powerful hardware future researchers could experiment with a higher number of folds in the cross-validation, or consider more parameters to tune in the hyperparameter tuning section.
Other technical limitations of this thesis are linked to the dataset, for example, having a bigger dataset could provide more data and train better algorithms.
Future researchers could use this research to compare the results with a more recent dataset, taking into account that the span of years considered for the songs in this thesis is from 1921 to 2020. This could help them compare their results with those provided by this research.
One possible idea for future research could be to implement the findings of this thesis in other music streaming platforms, considering how they evaluate whether a song is popular and whether they use different variables than those used by Spotify.

6. Conclusion

The study of the predictive abilities of machine learning algorithms for song popularity brought valuable insights. The Random Forest, Support Vector Machine, and Neural Network models were each employed to unravel the intricate mechanisms of what makes a song resonate with audiences.
Overall, F1 scores for class 1 of all three models were around 80%, indicative of their general competence in predicting the binary popularity variable. Despite the minor variations in their performance, the models displayed comparable results, confirming the notion that each approach contributes valuable predictive capabilities.
The Neural Network, while not drastically outperforming the other models, gave results closely similar to those of the Random Forest and SVM. This suggests that while the Neural Network may not be inherently more accurate for this specific task, it still offers valuable insights into predicting the popularity of songs.

References

1.17. neural network models (supervised). scikit. (n.d.). https://scikit-learn.org/stable/modules/neural_networks_supervised.html#:~:text=Multi%2Dlayer%20Perceptr on%20(MLP),number%20of%20dimensions%20for%20output.

160k Spotify songs from 1921 to 2020 (Sorted). (2022, September 17). Kaggle. https://www.kaggle.com/datasets/fcpercival/160k-spotify-songs-sorted

8.3. parallelism, resource management, and configuration. scikit. (n.d.-b). https://scikit-learn.org/stable/computing/parallelism.html

Araujo, C. V. S. (2020). A Model for Predicting Music Popularity on Spotify. Recall, 50, 173-90

Ay, Y. E. (2021). Spotify dataset 1921-2020, 160k+ tracks.

Bertin-Mahieux, T., Ellis, D. P., Whitman, B., & Lamere, P. (2011). The million song dataset.

De La Rosa K, RPubs - Machine Learning - Spotify Audio Features and Music Genres. (2021). https://rpubs.com/kdrdatascience/862303#:~:text=Danceability%3A%20A%20measure%20from%200.0,a%20track%20is%20more%20danceable

Dhanaraj, R., & Logan, B. (2005, September). Automatic Prediction of Hit Songs. In Ismir (pp. 488-491).

Essa, Y., Usman, A., Garg, T., & Singh, M. K. (2022). Predicting the Song Popularity Using Machine Learning Algorithm.

Gardner, M. W., & Dorling, S. R. (1998). Artificial neural networks (the multilayer perceptron)—a review of applications in the atmospheric sciences. Atmospheric environment, 32(14-15), 2627-2636.

Harris, C. R., Millman, K. J., Van Der Walt, S. J., Gommers, R., Virtanen, P., Cournapeau, D., ... & Oliphant, T. E. (2020). Array programming with NumPy. Nature, 585(7825), 357- 362.

Harris, C.R., Millman, K.J., van der Walt, S.J., Gommers, R., Virtanen, P., Cournapeau, D., Wieser, E., Taylor, J., Berg, S., Smith, N.J., Kern, R., Picus, M., Hoyer, S., van Kerkwijk, M.H., Brett, M., Haldane, A., del Río, J.F., Wiebe, M., Peterson, P., Gérard-Marchant, P., Sheppard, K., Reddy, T., Weckesser, W., Abbasi, H., Gohlke, C. and Oliphant, T.E. (2020) Array Programming with NumPy. Nature, 585, 357-362.

Herremans D, Martens D and Sörensen K, "Dance hit song prediction," Journal of New Music Research, vol. 43, no. 3, pp. 291–302, 2014. [Online]. Available: https://doi.org/10.1080/09298215.2014.881888

Hesmondhalgh, D. & Meier, L. (2015) "Popular music, independence and the concept of the alternative in contemporary capitalism", in Media Independence, eds. J. Bennet & N. Strange, Routledge, Abingdon and New York.

Hsu, C. W., Chang, C. C., & Lin, C. J. (2003). A practical guide to support vector classification.

Hunter, J. D. (2007). Matplotlib: A 2D graphics environment. Computing in science & engineering, 9(03), 90-95.

IFPI (2023) IFPI Global Music Report 2023, IFPI https://www.ifpi.org/wp-content/uploads/2020/03/Global_Music_Report_2023_State_of_the_Industry.pdf

Liu, Y., Wang, Y., & Zhang, J. (2012). New machine learning algorithm: Random forest. In Information Computing and Applications: Third International Conference, ICICA 2012, Chengde, China, September 14-16, 2012. Proceedings 3 (pp. 246-252). Springer Berlin Heidelberg.

Ma, Z., Sun, A., & Cong, G. (2013). On predicting the popularity of newly emerging hashtags in twitter. Journal of the American Society for Information Science and Technology, 64(7), 1399-1410.

Martín-Gutiérrez, D., Peñaloza, G. H., Belmonte-Hernández, A., & García, F. Á. (2020). A multimodal end-to-end deep learning architecture for music popularity prediction. IEEE Access, 8, 39361-39374.

McCulloch, W. S., & Pitts, W. (1943). A logical calculus of the ideas immanent in nervous activity. The bulletin of mathematical biophysics, 5, 115-133.

Mohapatra, N., Shreya, K., & Chinmay, A. (2020). Optimization of the random forest algorithm. In Advances in Data Science and Management: Proceedings of ICDSM 2019 (pp. 201-208). Springer Singapore.

O'Dair M & Fry A, (2020) Beyond the black box in music streaming: the impact of recommendation systems upon artists, Popular Communication, 18:1, 65-77, DOI: 10.1080/15405702.2019.1627548

Ogden, J. R., Ogden, D. T., & Long, K. (2011). Music marketing: A history and landscape. Journal of Retailing and Consumer Services, 18(2), 120–125. https://doi.org/10.1016/j.jretconser.2010.12.002

Otuokere, T. O., Imoize, A. L., & Atayero, A. A. A. (2021). Analysis of sonic effects of music from a comprehensive datasets on audio features. ELEKTRIKA-Journal of Electrical Engineering, 20(1), 43-53.

Pareek, P., Shankar, P., Pathak, M. P., & Sakariya, M. N. (2022). Predicting music popularity using machine learning algorithm and music metrics available in spotify. J. Dev. Econ. Manag. Res. Stud. JDMS, 9, 10-19.

Pedregosa, F., Varoquaux, G., Gramfort, A., Michel, V., Thirion, B., Grisel, O., ... & Duchesnay, É. (2011). Scikit-learn: Machine learning in Python. the Journal of machine Learning research, 12, 2825-2830.

McKinney, W. (2010, June). Data structures for statistical computing in python. In Proceedings of the 9th Python in Science Conference (Vol. 445, No. 1, pp. 51-56).

OpenAI. (2023). ChatGPT (Feb 13 version) [Large language model]. https://chat.openai.com

Raza, A. H., & Nanath, K. (2020, July). Predicting a Hit Song with Machine Learning: Is there an apriori secret formula?. In 2020 International Conference on Data Science, Artificial Intelligence, and Business Analytics (DATABIA) (pp. 111-116). IEEE.

Singhi, A., & Brown, D. G. (2015). Can song lyrics predict hits. In Proceedings of the 11th International Symposium on Computer Music Multidisciplinary Research (pp. 457-471).

Trzciński, T., & Rokita, P. (2017). Predicting popularity of online videos using support vector regression. IEEE Transactions on Multimedia, 19(11), 2561-2570.

Waskom, M. L. (2021). Seaborn: statistical data visualization. Journal of Open Source Software, 6(60), 3021.

Web API Reference | Spotify for Developers. (n.d.) https://developer.spotify.com/documentation/web-api/reference/get-audio-features

Appendix 1

Following is the link to download the 160000 dataset created by Ay.

https://github.com/gabminamedez/spotify-data/blob/master/data.csv

Appendix 2

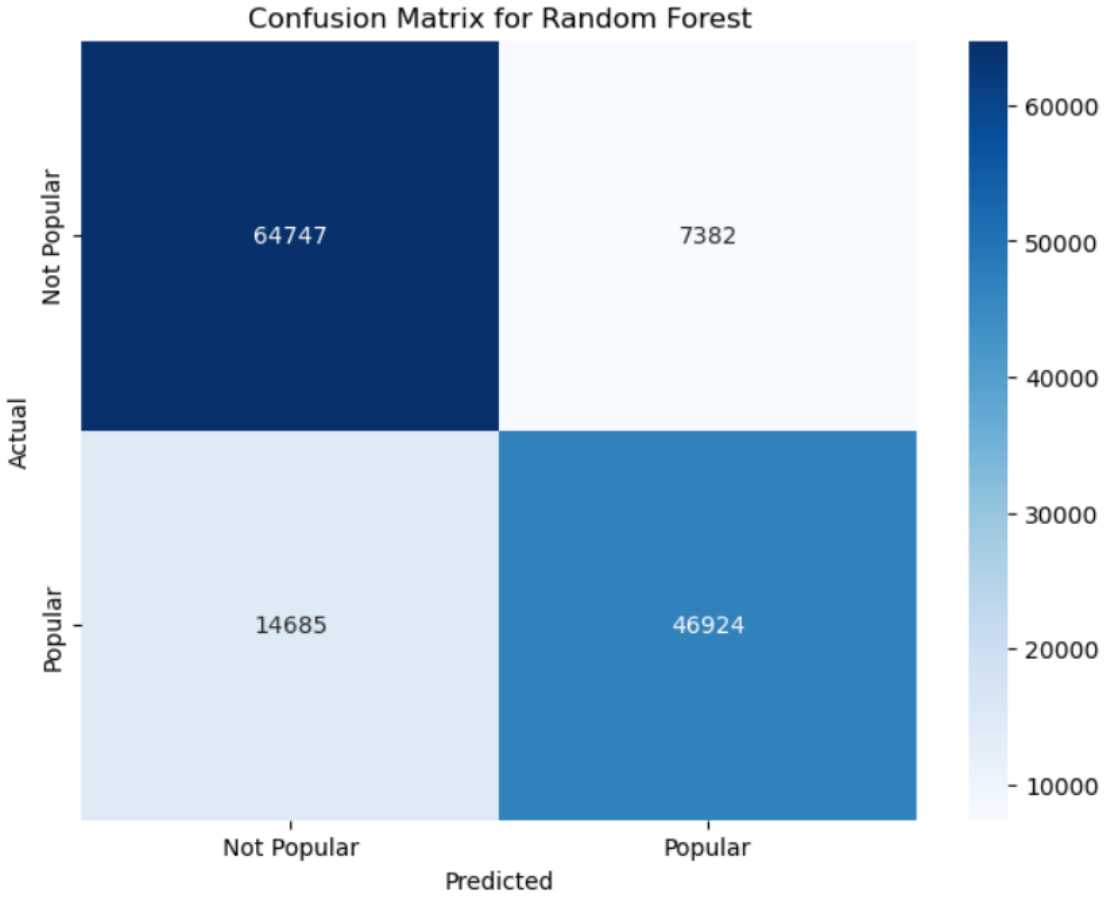

Figure 5: Confusion matrix for the Random Forest model

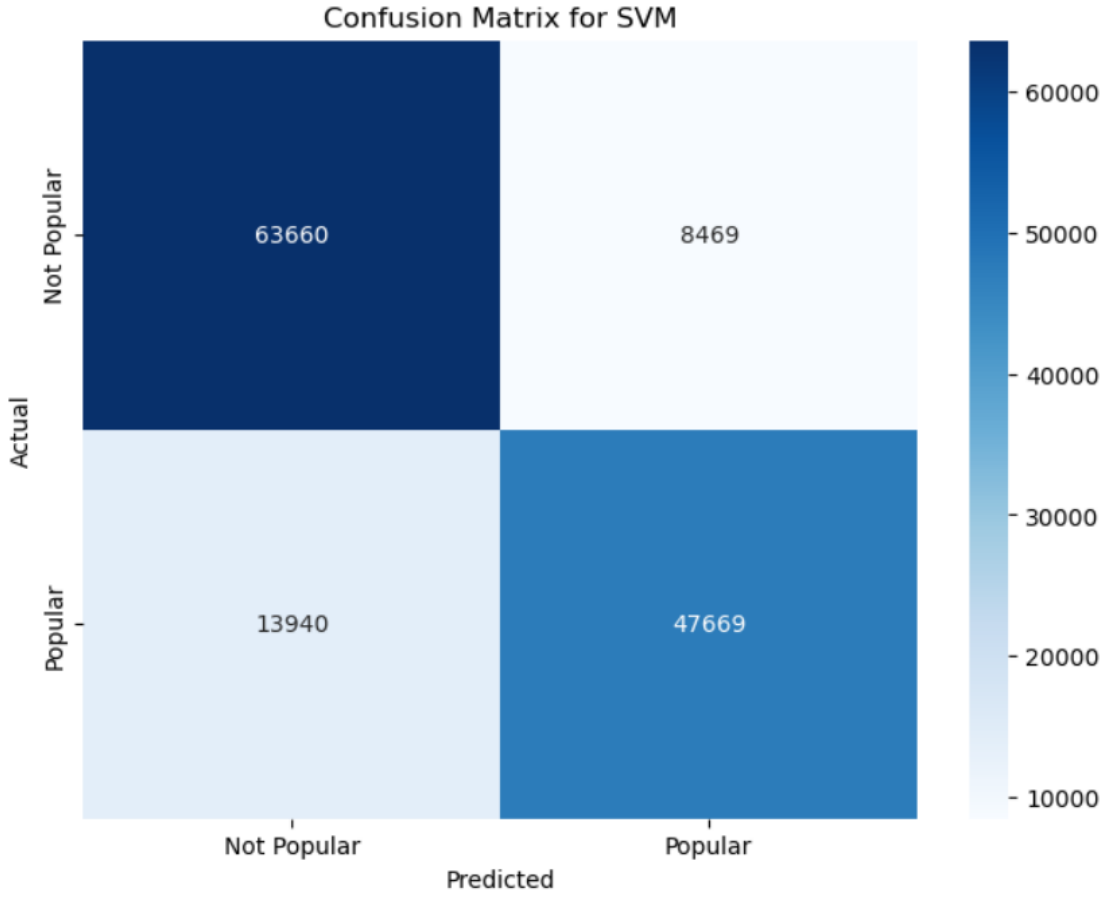

Figure 6: Confusion matrix for the Support Vector Machine model

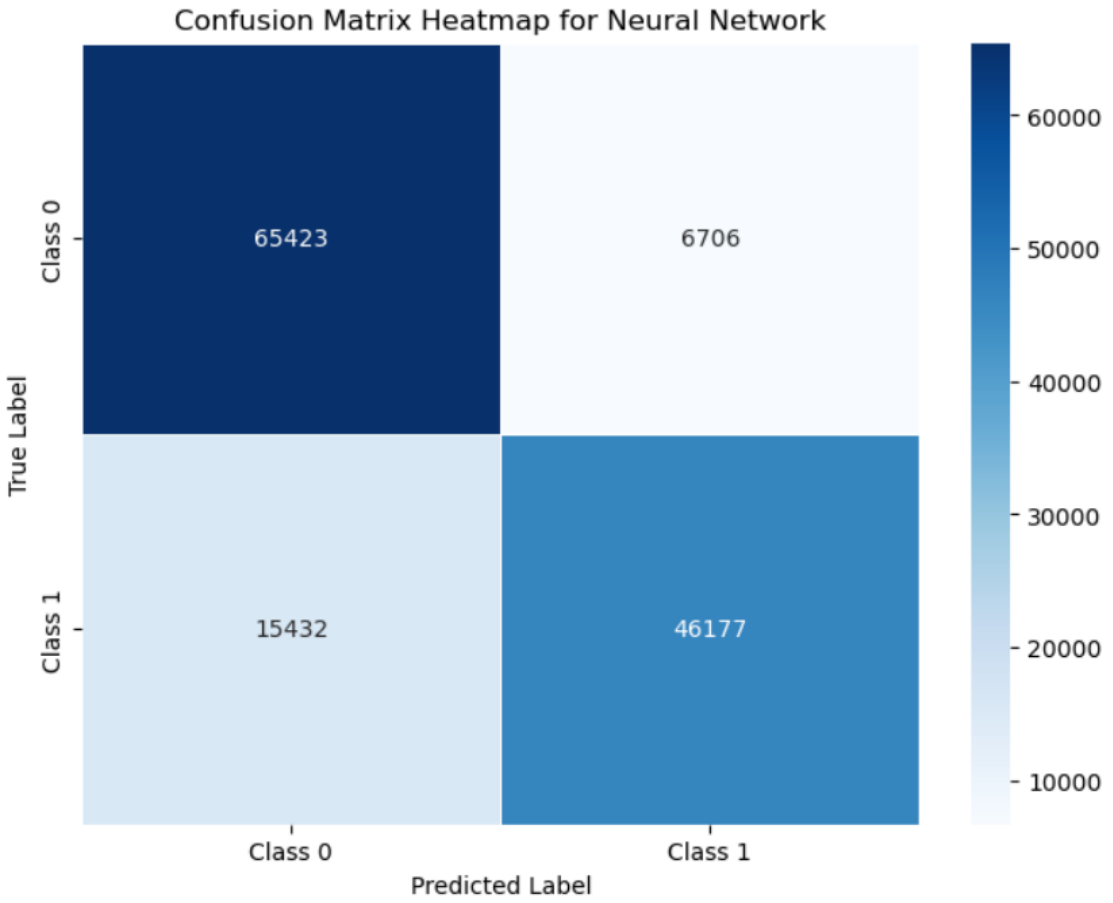

Figure 7: Confusion matrix for the Neural Network model